Part 2 — Python AI Libraries — The One Question, Explained

The one question: write ONE program that uses all four libraries of Part 2 of the Ultimate AI Book — NumPy, pandas, Matplotlib, and scikit-learn. Questions 301 to 600. Below, each important piece of the code is explained in plain words, with the code itself embedded exactly as written.

1. The imports — bringing in the four libraries

The program starts by bringing in the four libraries of Part 2 and giving the long names short nicknames. NumPy becomes np, pandas becomes pd, and the chart maker from Matplotlib becomes plt. The fourth line reaches inside scikit-learn and brings out one ready-made learning model named LinearRegression. Every later line uses the short names.

```
import numpy as np           # bring in NumPy and give it the short name np
import pandas as pd          # bring in pandas and give it the short name pd
import matplotlib.pyplot as plt   # bring in the chart maker and call it plt
from sklearn.linear_model import LinearRegression   # bring in one ready-made learning model
```

2. NumPy — fast math on a table of numbers

NumPy holds the four scores in one array: 40, 55, 70 and 85. An array is one fast table of numbers, all of the same kind, and NumPy can do math on the whole table in a single move instead of number by number. The mean of the four scores is 62.5, and the biggest score is 85.

```
scores = np.array([40, 55, 70, 85])        # a NumPy array: one fast table of numbers, all the same type
print("Mean score:", scores.mean())        # NumPy does the math on the whole table at once: the mean is 62.5
print("Top score:", scores.max())          # the biggest number in the table: 85
```

3. pandas — a table with named columns

pandas builds a proper table named students. It has two columns: Name holds Sara, John, Alicia and Abel, and Score holds the same NumPy array from the step above, so each name sits beside its score. The second line filters the table and keeps only the rows where the score reaches 60. Two rows survive: Alicia with 70 and Abel with 85.

```
students = pd.DataFrame({"Name": ["Sara", "John", "Alicia", "Abel"], "Score": scores})   # a pandas table: names in one column, scores in the other
print(students[students["Score"] >= 60])   # show only the rows where the score reaches 60: Alicia and Abel
```

4. Matplotlib — turning the table into a chart

Matplotlib draws the table as a bar chart: one bar per student, and each bar is as tall as that student's score. The second line saves the chart as a picture file named scores_chart.png instead of only showing it on a screen, so the chart can be kept, sent, or printed later.

```
plt.bar(students["Name"], students["Score"])   # draw one bar per student, as tall as the score
plt.savefig("scores_chart.png")   # save the chart as a picture file instead of only showing it
```

5. scikit-learn — a model that learns the pattern

The last library does the learning. The clues are study hours: 1, 2, 3 and 4 hours, one row per student, and the answers the model must learn are the same four scores. A fresh LinearRegression model starts empty and untrained. The fit line trains it by showing it the hours together with the real scores until it finds the pattern: each extra hour adds 15 points. Then the program asks the trained model to predict the score for 5 study hours, and the model answers 100.0.

```
hours = np.array([[1], [2], [3], [4]])   # study hours: the clues the model learns from, one row per student
model = LinearRegression()   # create the model, still empty and untrained
model.fit(hours, scores)     # train it: show it the hours together with the real scores until it finds the pattern
print("Predicted score for 5 study hours:", model.predict([[5]]))   # ask the trained model to guess the score for 5 hours: 100.0
```

The full output

When the whole program runs from top to bottom, this is exactly what it produces.

```
Mean score: 62.5
Top score: 85
Passing rows: Alicia 70, Abel 85
Predicted score for 5 study hours: 100.0
Plus one saved file: scores_chart.png, the bar chart picture
```